

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ТЕЛЕКОММУНИКАЦИЙ ИМ. ПРОФ. М.А. БОНЧ-БРУЕВИЧА»
(СПбГУТ)

АРХАНГЕЛЬСКИЙ КОЛЛЕДЖ ТЕЛЕКОММУНИКАЦИЙ
ИМ. Б.Л. РОЗИНГА (ФИЛИАЛ) СПбГУТ
(АКТ (ф) СПбГУТ)

КУРСОВОЙ ПРОЕКТ

НА ТЕМУ

РАЗРАБОТКА ПОДСИСТЕМЫ

«КУРСЫ ШВЕДСКОГО ЯЗЫКА»

Л109. 25КП01. 03 ПЗ

(Обозначение документа)

МДК.02.01 Технология разработки

программного обеспечения

Студент	ИСПП-35	08.12.2025	А.В. Богачева
	(Группа)	(Подпись)	(И.О. Фамилия)
Преподаватель		09.12.2025	Ю.С. Маломан
	(Подпись)	(Дата)	(И.О. Фамилия)

Архангельск 2025

СОДЕРЖАНИЕ

Перечень сокращений и обозначений.....	3
Введение.....	4
1 Анализ и разработка требований.....	7
1.1 Назначение и область применения.....	7
1.2 Постановка задачи	7
1.3 Выбор состава программных и технических средств	8
2 Проектирование программного обеспечения	11
2.1 Проектирование интерфейса пользователя.....	11
2.2 Разработка архитектуры программного обеспечения.....	11
2.3 Проектирование базы данных	13
3 Разработка и интеграция модулей программного обеспечения.....	14
3.1 Разработка программных модулей.....	14
3.2 Реализация интерфейса пользователя.....	16
3.3 Разграничение прав доступа пользователей	17
3.4 Экспорт и импорт данных.....	18
4 Тестирование и отладка программного обеспечения.....	21
4.1 Структурное тестирование.....	21
4.2 Функциональное тестирование	22
5 Инструкция по эксплуатации программного обеспечения	24
5.1 Установка программного обеспечения.....	24
5.2 Инструкция по работе	25
Заключение	29
Список использованных источников	30

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящем курсовом проекте применяют следующие сокращения и обозначения.

БД – база данных

ОС – операционная система

ПО – программное обеспечение

СУБД – система управления базами данных

.NET – программная платформа Microsoft

API – интерфейс программирования приложения

ASP.NET – активные страницы сервера для .NET

DTO – объект передачи данных

ERD – диаграмма сущность–связь

HTTP – протокол передачи гипертекста

JSON – текстовый формат обмена данными

JWT – веб-токен JSON

ВВЕДЕНИЕ

Веб-приложения являются одной из наиболее востребованных интернет-технологий современного мира, выступая ключевым инструментом взаимодействия между пользователями и цифровыми сервисами. Через веб-интерфейсы обеспечивается удобный доступ к информации, образовательным материалам, записи на курс, что делает их важнейшим элементом цифровой инфраструктуры.

В условиях активной цифровизации образования и роста интереса к изучению иностранных языков возрастает необходимость в создании удобных и адаптивных онлайн-платформ. Особенно это актуально для изучения менее распространённых языков, к которым относится шведский. Традиционных образовательных ресурсов по шведскому языку в русскоязычном сегменте Интернета недостаточно, а существующие решения нередко имеют ограниченный функционал или не обеспечивают должного уровня интерактивности.

Поэтому разработка подсистемы «Курсы шведского языка» является важным шагом в повышении доступности и эффективности изучения языка. Создание специализированного онлайн-ресурса позволит систематизировать учебные материалы, обеспечить удобное взаимодействие между пользователем и обучающей платформой, автоматизировать процесс подачи и проверки заданий, а также сделать обучение более гибким и персонализированным.

Внедрение описанной подсистемы позволит значительно улучшить качество образовательного процесса, повысить мотивацию пользователей, сократить временные затраты на поиск учебных материалов и обеспечить единое пространство для изучения шведского языка. Таким образом, актуальность проекта обусловлена необходимостью создания современного, удобного и функционального инструмента, который позволит удовлетворить

растущий спрос на онлайн-обучение и расширить доступ к изучению шведского языка.

Целью курсового проектирования является разработка комплексного веб-решения для организации онлайн-обучения шведскому языку, обеспечивающего создание, редактирование и удаление курсов, лекций и тестов. Реализация проекта позволяет повысить эффективность процесса обучения, минимизировать организационные трудности.

Для достижения поставленной цели требуется решить следующие задачи:

- провести сбор требований целевой аудитории,
- проанализировать информационные источники по предметной области,
- спроектировать архитектуру веб-приложения,
- спроектировать диаграмму вариантов использования приложения,
- выбрать состав программных и технических средств для реализации приложения,
- спроектировать БД,
- спроектировать интерфейс веб-приложения,
- создать БД в выбранной СУБД,
- разработать API для некоторых функций приложения,
- реализовать разграничение прав доступа пользователей,
- разработать интерфейс веб-приложения,
- разработать веб-приложение,
- реализовать работу веб-приложения с сервером БД при помощи REST API,
- выполнить структурное тестирование ПО,
- выполнить функциональное тестирование ПО,
- разработать программную и эксплуатационную документацию.

В результате выполнения поставленных задач будет разработано веб-приложение, обеспечивающее автоматизацию учебного процесса в языковой школе.

1 Анализ и разработка требований

1.1 Назначение и область применения

Подсистема «Курсы шведского языка» предназначена для автоматизации процессов обучения, хранения учебных материалов. Система обеспечивает удобный доступ к лекциям и тестам, сокращает время поиска информации, повышает качество учебного процесса.

1.2 Постановка задачи

Необходимо разработать веб-приложение, которое предоставит доступ к следующей функциональности:

- регистрации и авторизации пользователей;
- созданию, редактированию и удалению информации о курсах;
- прохождению тестов и получение мгновенного результата;
- формированию списка имеющихся курсов в формате *.pdf.

К веб-приложению предъявляются следующие эксплуатационные требования:

- интуитивно понятный интерфейс пользователя,
- авторизация и разграничение прав доступа,
- устойчивость при сбоях сети и корректное сохранение данных.

Неавторизованный пользователь может просматривать на главной странице информацию о самых популярных курсах и страницу входа, а также может зарегистрироваться.

Студент имеет возможность записываться на курсы, просматривать информацию о них, просматривать лекции курса, на который записан, проходить тесты выбранного курса.

Преподаватель может создавать, удалять и редактировать курсы.

Администратор обладает полным контролем: может удалять информацию о курсах, и пользователях, редактировать и создавать курсы, формировать список курсов в формате *.pdf, импортировать новые курсы из формата *.csv, *.xls.

На рисунке 1 изображена диаграмма вариантов использования приложения.



Рисунок 1 – Диаграмма вариантов использования

1.3 Выбор состава программных и технических средств

Согласно цели проекта требуется создать веб-приложение «Курсы шведского языка» для автоматизации процесса обучения.

Разработка веб-приложения будет осуществляться на языке C# с использованием фреймворка ASP.NET Core, который обеспечивает четкую архитектуру и поддержку MVC-паттерна для организации кода.

Разработка клиентской части веб-приложения будет реализована на языке JavaScript с использованием библиотеки React, которая обеспечивает эффективное управление пользовательским интерфейсом.

В качестве СУБД выбрана MySQL, отличающаяся стабильностью, обширными возможностями работы с данными и высокой производительностью. Она хорошо интегрируется с .NET-приложениями.

Веб-сервером для публикации и маршрутизации приложения выбран Caddy, который будет эффективно обрабатывать запросы и обеспечивать стабильную работу приложения.

Разработка будет выполняться в Visual Studio, интегрированной среде разработки, поддерживающей ASP.NET Core, работу с Docker и MySQL. Visual Studio предлагает встроенные средства отладки и управления зависимостями.

Для функционирования системы на стороне сервера достаточны следующие программные и технические средства:

- ОС Ubuntu версии 18.04 и выше;
- сервер БД: MySQL не ниже 8.0.40;
- процессор с 2 ядрами по 2 ГГц;
- оперативная память объемом 2 ГБ;
- ПО для конфигурирования, управления и администрирования сервера БД: MySQL Workbench последней версии;
- ПО для работы API: Docker, docker-compose, React, Node.js, ASP.NET Core SDK.

Для функционирования системы на стороне клиента достаточны следующие программные и технические средства:

- браузер с поддержкой HTML5 и CSS3;
- процессор с частотой 1 ГГц или быстрее;

- оперативная память в объеме 1 ГБ и выше;
- свободное место в хранилище 40 МБ.

2 Проектирование программного обеспечения

2.1 Проектирование интерфейса пользователя

В рамках разработки веб-приложения «Курсы шведского языка» спроектирован интерфейс пользователя при помощи Figma. Wireframe позволяют наглядно увидеть структуру приложения, его основные элементы и функциональность.

Wireframe формы создания курса показаны рисунке 2.

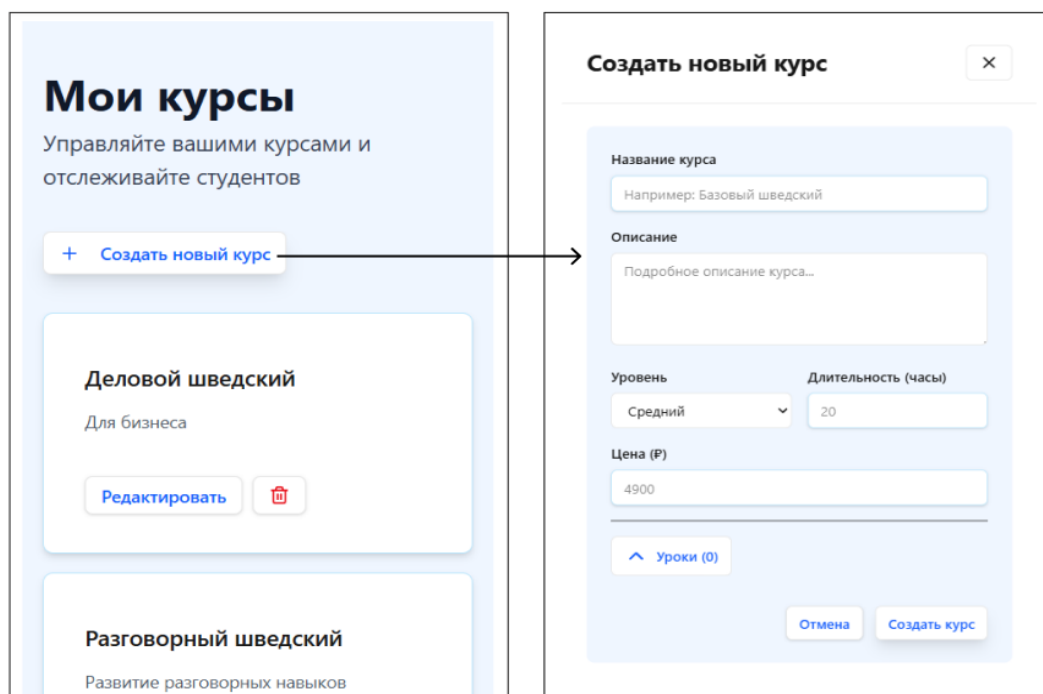


Рисунок 2 – Figma. Wireframe формы создания курса.

2.2 Разработка архитектуры программного обеспечения

Приложение предназначено для автоматизации процесса обучения, хранения и обработки курсов шведского языка. Архитектура приложения построена на основе клиент-серверной модели и включает в себя следующие элементы: серверная часть приложения, веб-приложение, БД[3].

Взаимодействие между клиентской частью и сервером осуществляется через HTTP-запросы к API, построенному на ASP.NET Core. Обмен данными между клиентом и сервером производится в формате JSON для структурных данных.

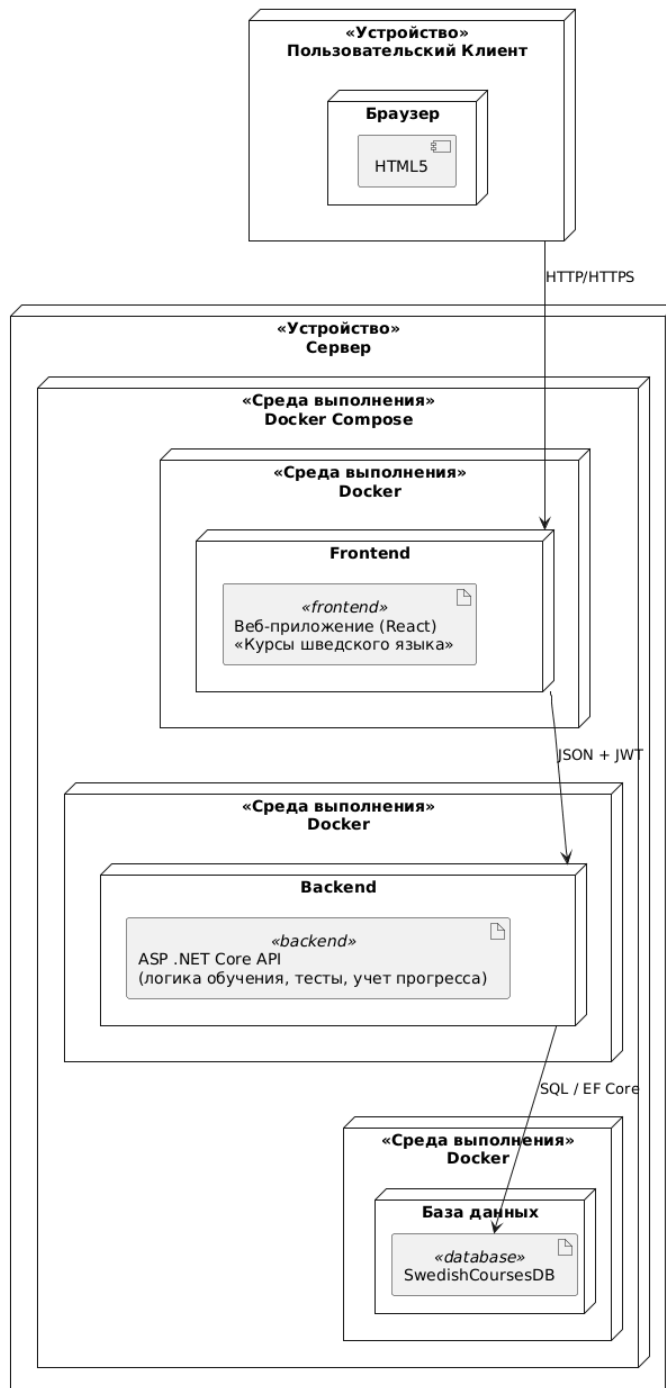


Рисунок 3 – Диаграмма развёртывания

2.3 Проектирование базы данных

Требуется разработать БД для хранения и управления данными. На рисунке 3 в виде ERD показана физическая модель БД, спроектированная с использованием MySQL Workbench.

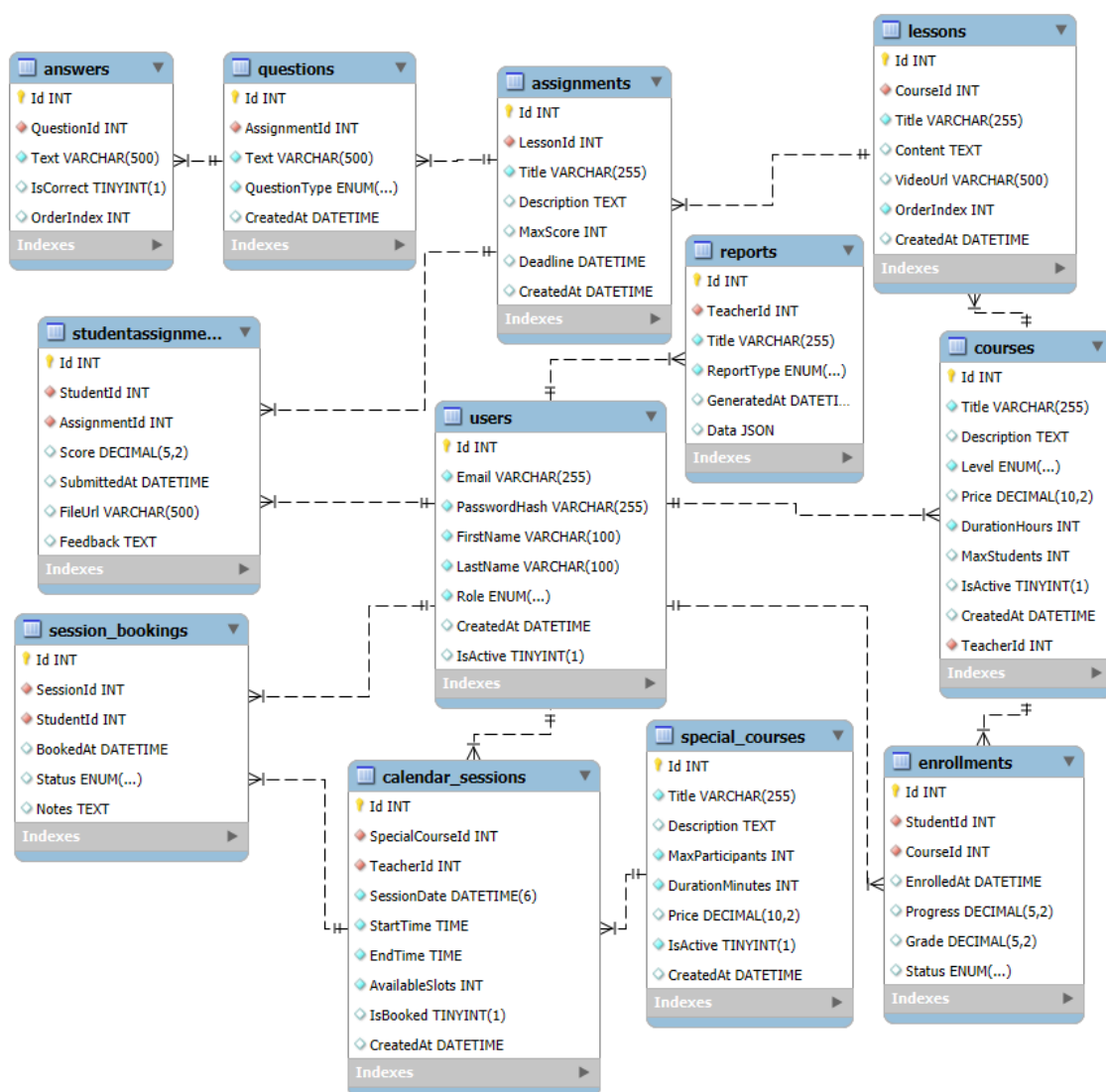


Рисунок 4 – MySQL Workbench. Физическая модель БД

3 Разработка и интеграция модулей программного обеспечения

3.1 Разработка программных модулей

В рамках курсового проектирования разработано веб-приложение, включающее серверную часть на ASP.NET Core и клиентский интерфейс на React, созданные в среде Visual Studio [2].

Для реализации получения данных о курсах используется встроенный механизм отправки HTTP-запросов. Код метода для получения списка курсов представлен листингом 1.

Листинг 1 – Код метода GET для получения списка курсов

```
/// <summary>Получает список курсов с учетом фильтрации и
пагинации.</summary>
[HttpGet]
public async Task<ActionResult<PagedResponseDto<CourseDto>>>
GetCourses(
    [FromQuery] PaginationDto pagination,
    [FromQuery] string? level = null,
    [FromQuery] string? search = null)
{
    // Формируем запрос
    var query = _context.Courses
        .Include(c => c.Teacher)
        .AsQueryable();

    // Фильтр по уровню
    if (!string.IsNullOrEmpty(level))
        query = query.Where(c => c.Level == level);

    // Поиск по названию/описанию
    if (!string.IsNullOrEmpty(search))
        query = query.Where(c =>
            (c.Title ?? "").Contains(search) ||
            (c.Description ?? "").Contains(search));

    // Общее количество перед пагинацией
    var totalCount = await query.CountAsync();
```

```

// Выборка страниц
var items = await query
    .OrderBy(c => c.Title)
    .Skip(pagination.Skip)
    .Take(pagination.Take)
    .Select(c => new CourseDto
    {
        Id = c.Id,
        Title = c.Title ?? "",
        Description = c.Description ?? "",
        Level = c.Level ?? "",
        Price = c.Price,
        DurationHours = c.DurationHours,
        TeacherId = c.TeacherId,
        TeacherName = c.Teacher.FirstName + " " +
c.Teacher.LastName
    })
    .ToListAsync();

// Формирование ответа
return Ok(new PagedResponseDto<CourseDto>
{
    Items = items,
    TotalCount = totalCount,
    Page = pagination.Page,
    PageSize = pagination.PageSize
});
}

```

Реализация загрузки данных курса в React [4] представлена листингом 2.

Листинг 2 – Код метода отображения данных о курсе на странице «Курсы»

```

// Получение данных о курсе и отображение результата
export const CourseDetailsPage = () => {
    const { id } = useParams();
    const courseId = Number(id);

    // GET-запрос к API
    const { data: course, isLoading, error } =
useGetCourseById(courseId);

    if (isLoading) return <p>Загрузка...</p>;
    if (error || !course) return <p>Ошибка загрузки курса</p>;

    // Отображение данных, полученных с сервера
    return (
        <div>
            <h1>{course.title}</h1>

```

```

    <p>Уровень: {course.level}</p>
    <p>Длительность: {course.durationHours} ч.</p>
    <p>Преподаватель: {course.teacherName}</p>
    <p>Стоимость: ${course.price}</p>
  </div>
);
};

```

3.2 Реализация интерфейса пользователя

Для отображения информации о курсе и возможности записи на него разработан компонент `CourseDetailsPage.tsx` [4], код которого представлен листингом 3.

Листинг 3 – Код страницы `CourseDetailsPage.tsx`

```

// Компонент отображает информацию о курсе и кнопку записи
export const CourseDetailsPage = () => {
  const { id } = useParams();
  const courseId = Number(id);
  const { user } = useAuth();

  // Получение данных курса с сервера
  const { data: course, isLoading, error } =
    useGetCourseById(courseId);
  const { data: enrollments } =
    useGetStudentEnrollments(user?.id || 0, { page: 1, pageSize: 100
    });
  const enrollMutation = useEnrollStudent();

  const isAlreadyEnrolled = enrollments?.items?.some(e =>
    e.courseId === courseId);

  const handleEnroll = async () => {
    if (!user) return;
    await enrollMutation.mutateAsync({ courseId, studentId:
    user.id });
  };

  if (isLoading) return <p>Загрузка...</p>;
  if (error || !course) return <p>Курс не найден</p>;

  return (
    <div>
      <h1>{course.title}</h1>
      <p>Уровень: {course.level}</p>

```



```

        <p>Длительность: {course.durationHours} ч.</p>
        <p>Преподаватель: {course.teacherName}</p>
        <p>Стоимость: ${course.price}</p>

        <button onClick={handleEnroll}
disabled={enrollMutation.isPending || isAlreadyEnrolled}>
            {isAlreadyEnrolled ? 'Вы записаны на курс' : 'Записаться
на курс'}
        </button>
    </div>
    );
};

```

3.3 Разграничение прав доступа пользователей

В приложении разработано разграничение прав доступа пользователей с помощью JWT и безопасное хранение паролей с помощью BCrypt [5]. Для этого в приложении реализована авторизация.

Код авторизации с помощью JWT представлен листингом 4.

Листинг 4 – Код авторизации пользователей

```

// Контроллер аутентификации, обеспечивающий регистрацию и вход
пользователей, генерация JWT
Route("api/[controller]")
public class AuthController : ControllerBase
{
    private readonly ApplicationDbContext _context;
    private readonly TokenService _tokenService;

    public AuthController(ApplicationDbContext context,
TokenService tokenService)
    {
        _context = context;
        _tokenService = tokenService;
    }

    // Регистрация пользователя
    [HttpPost("register")]
    public async Task<ActionResult<AuthResponseDto>>
Register(RegisterDto dto)
    {
        if (await _context.Users.AnyAsync(u => u.Email ==
dto.Email))

```

```

        return BadRequest("Пользователь с таким email уже
существует");

        var user = new User
        {
            Email = dto.Email,
            FirstName = dto.FirstName,
            LastName = dto.LastName,
            Role = dto.Role,
            PasswordHash =
BCrypt.Net.BCrypt.HashPassword(dto.Password),
            CreatedAt = DateTime.UtcNow
        };

        _context.Users.Add(user);
        await _context.SaveChangesAsync();
        var token = _tokenService.CreateToken(user);
        return new AuthResponseDto { User = user, Token = token
    };
}

// Вход пользователя
[HttpPost("login")]
public async Task<ActionResult<AuthResponseDto>>
Login(LoginDto dto)
{
    var user = await _context.Users.FirstOrDefaultAsync(u =>
u.Email == dto.Email);
    if (user == null ||
!BCrypt.Net.BCrypt.Verify(dto.Password, user.PasswordHash))
        return Unauthorized("Неверный email или пароль");
    var token = _tokenService.CreateToken(user);
    return new AuthResponseDto { User = user, Token = token
};
}

```

3.4 Экспорт и импорт данных

В подсистеме реализован импорт и экспорт данных курсов. Для безопасности права на импорт данных есть только у пользователей с ролью администратора. Код контроллера представлен листингом 5.

Листинг 5 – Контроллер импорта и экспорта курсов

```

// Метод импорт и экспорта данных
[ApiController]

```

```

[Route("api/import-export")]
public class ImportExportController : ControllerBase
{
    private readonly ApplicationDbContext _context;
    private readonly ILogger<ImportExportController> _logger;
    private readonly PdfExportService _pdfService;

    public ImportExportController(ApplicationDbContext context,
        ILogger<ImportExportController> logger, PdfExportService
        pdfService)
    {
        _context = context;
        _logger = logger;
        _pdfService = pdfService;
    }

    // Экспорт всех курсов в PDF
    [HttpGet("export/courses/pdf")]
    public async Task<IActionResult> ExportAllCoursesPdf()
    {
        var courses = await _context.Courses.Include(c =>
        c.Teacher).OrderBy(c => c.Title).ToListAsync();
        var pdfBytes =
        _pdfService.GenerateCoursesListPdf(courses);
        return File(pdfBytes, "application/pdf",
        $"courses_catalog_{DateTime.Now:yyyyMMdd}.pdf");
    }

    // Импорт курсов из Excel (только для администраторов)
    [HttpPost("import/courses/excel")]
    [Authorize(Roles = "Admin")]
    public async Task<ActionResult>
    ImportCoursesFromExcel(IFormFile file)
    {
        if (file == null || file.Length == 0)
            return BadRequest(new { message = "Файл Excel не
            загружен" });

        var importedCourses = new List<Course>();
        using var stream = new MemoryStream();
        await file.CopyToAsync(stream);
        using var workbook = new XLWorkbook(stream);
        var worksheet = workbook.Worksheet(1);

        var rows = worksheet.RangeUsed()?.RowsUsed().Skip(1); //
        пропустить заголовок
        if (rows == null) return Ok(new { message = "Нет данных
        для импорта", importedCount = 0 });

        foreach (var row in rows)
        {
            var title = row.Cell(1).GetValue<string>()?.Trim();

```

```

        var description =
row.Cell(2).GetValue<string>()?.Trim();
        var level = row.Cell(3).GetValue<string>()?.Trim()
?? "A1";
        var price = row.Cell(4).GetValue<decimal>();
        var durationHours = row.Cell(5).GetValue<int>();
        var teacherEmail =
row.Cell(7).GetValue<string>()?.Trim();

        if (string.IsNullOrEmpty(title) ||
string.IsNullOrEmpty(teacherEmail)) continue;

        var teacher = await
_context.Users.FirstOrDefaultAsync(u => u.Email ==
teacherEmail);
        if (teacher == null) continue;

        var existingCourse = await
_context.Courses.FirstOrDefaultAsync(c => c.Title == title);
        if (existingCourse != null) continue;

        importedCourses.Add(new Course
        {
            Title = title,
            Description = description,
            Level = level,
            Price = price,
            DurationHours = durationHours,
            IsActive = true,
            CreatedAt = DateTime.UtcNow,
            TeacherId = teacher.Id
        });
    }

    if (importedCourses.Any())
    {
        await
_context.Courses.AddRangeAsync(importedCourses);
        await _context.SaveChangesAsync();
    }

    return Ok(new
    {
        message = $"Импортировано {importedCourses.Count}
курсов",
        importedCount = importedCourses.Count,
        courses = importedCourses.Select(c => new { c.Id,
c.Title, c.Level }).ToList()
    });
}
}

```

4 Тестирование и отладка программного обеспечения

4.1 Структурное тестирование

Во время выполнения курсового проекта проведено структурное тестирование контролера CourseControllerTests[1]. Код unit-теста метода создания курса с уже занятым названием представлен листингом 6.

Листинг 6 – Код unit-теста для CourseControllerTests

```
// Код класса CourseControllerTests
public class CourseControllerTests
{
    // Создаём in-memory контекст
    private ApplicationDbContext GetDbContext()
    {
        var options = new
DbContextOptionsBuilder<ApplicationDbContext>()
        .UseInMemoryDatabase(Guid.NewGuid().ToString()) //
уникальная БД на каждый тест
        .Options;

        return new ApplicationDbContext(options);
    }

    // Тест успешного создания курса
    [Fact]
    public async Task CreateCourse_Success()
    {
        var context = GetDbContext();
        var controller = new CourseController(context);

        var dto = new CreateCourseDto
        {
            Title = "Шведский A2",
            Description = "Курс для изучающих на уровне A2",
            Level = "Intermediate",
            Price = 1200,
            DurationHours = 40
        };

        var result = await controller.CreateCourse(dto);

        // Проверяем, что вернулся OK
        var ok = Assert.IsType<OkObjectResult>(result);
    }
}
```

```

        var course = Assert.IsType<Course>(ok.Value);

        Assert.Equal("Шведский A2", course.Title);
        Assert.Equal("Intermediate", course.Level);
        Assert.Equal(1200, course.Price);
    }

    // Тест попытки создать курс с существующим названием
    [Fact]
    public async Task CreateCourse_AlreadyExists()
    {
        var context = GetDbContext();

        // Добавляем курс вручную
        context.Courses.Add(new Course
        {
            Title = "Шведский A2",
            Description = "Описание",
            Level = "Intermediate",
            Price = 1200,
            DurationHours = 30
        });
        await context.SaveChangesAsync();

        var controller = new CourseController(context);

        var dto = new CreateCourseDto
        {
            Title = "Шведский A2", // дубль
            Description = "Новое описание",
            Level = "Intermediate",
            Price = 1500,
            DurationHours = 40
        };

        var result = await controller.CreateCourse(dto);

        // Ожидаем BadRequest
        Assert.IsType<BadRequestObjectResult>(result);
    }

```

4.2 Функциональное тестирование

Во время функционального тестирования проведена проверка подсистемы методом «черного ящика», результаты тестирования представлены в таблице 1.

Таблица 1 – Набор тестов для приложения

Действие	Ожидаемый результат	Фактический результат
Нажать на кнопку «Войти» в верхнем правом углу экрана. Ввести в поле email «admin@school.com» и в поле пароля «temp123»	Над кнопкой «Авторизоваться» отображается надпись «Неверный логин или пароль»	Совпадает с ожидаемым
Нажать на кнопку «Войти» в верхнем правом углу экрана. Ввести в поле email «teacher@school.com» и в поле пароля «Temp1223», нажать на кнопку «Каталог курсов», на странице «Каталог курсов» из списка курсов выбрать «Бизнес шведский» и нажать на кнопку «Удалить»	Появляется окно с надписью «Курс “Бизнес шведский” успешно удален!»	Совпадает с ожидаемым
Нажать на кнопку «Войти» в верхнем правом углу экрана. Ввести в поле email «student2@school.com» и в поле пароля «tempMar123», нажать на кнопку «Каталог курсов», на странице «Каталог курсов» из списка курсов выбрать «Разговорный шведский» и нажать на кнопку «Записаться на курс»	Пользователь получает подтверждение о записи, кнопка меняется на «Вы записаны на курс»	Совпадает с ожидаемым
Нажать на кнопку «Войти» в верхнем правом углу экрана. Ввести в поле email «admin@school.com» и в поле пароля «tempAdmin123», нажать на кнопку «Импорт/Экспорт», на странице «Импорт/Экспорт» нажать на кнопку «Скачать в PDF»	Формируется PDF-файл со списком курсов	Совпадает с ожидаемым
Нажать на кнопку «Войти» в верхнем правом углу экрана. Ввести в поле email «admin@school.com» и в поле пароля «tempAdmin123», нажать на кнопку «Управление пользователями», на странице «Управление пользователями» и выбрать из списка студентов «Иван Данилов» для удаления, нажать на кнопку «Удалить»	Появляется окно с надписью «Пользователь “Иван данилов” успешно удален!»	Совпадает с ожидаемым

После проведения функционального тестирования можно сделать вывод, что приложение работает корректно.

5 Инструкция по эксплуатации программного обеспечения

5.1 Установка программного обеспечения

Для развёртывания БД нужно подключиться к серверу MySQL, с помощью MySQL Workbench и выполнить SQL-скрипт, находящийся в репозитории проекта.

Для установки серверной части требуется перейти в терминале в требуемую папку для API, клонировать репозиторий, создать файл appsettings.json по примеру файла appsettings.Development.json и в терминале использовать команду представленную листингом 7.

Листинг 7 – Код запуска API средствами docker-compose

```
# Код запуска API средствами docker-compose
docker-compose up -d
```

Фронтенд-приложение на React запускается из папки frontend после установки зависимостей: React, Vite, TypeScript, Tailwind CSS, Zustand, React Query.

Листинг 8 – Код установки зависимостей и запуск приложения

```
#установка зависимостей
npm install
#запуск
npm start
```


Для авторизации в приложении предусмотрены следующие учетные данные:

- логин: admin@school.com,
- пароль: Temp12333.

5.2 Инструкция по работе

Для авторизации требуется ввести учётные данные в поля ввода email и пароля и нажать на кнопку «Авторизоваться». Окно авторизации показано на рисунке 5.

Рисунок 5 – Школа шведского языка. Вид окна «Авторизация»

После авторизации пользователь с ролью «Администратор» перенаправляется на окно кабинета пользователя, где ему доступны страницы «Главная», «Управление курсами», «Управление пользователями» и «Импорт/Экспорт». По умолчанию в главном окне открыта страница «Главная», в которой он может просмотреть информацию о пользователе. Вид страницы «Главная» представлен на рисунке 6.

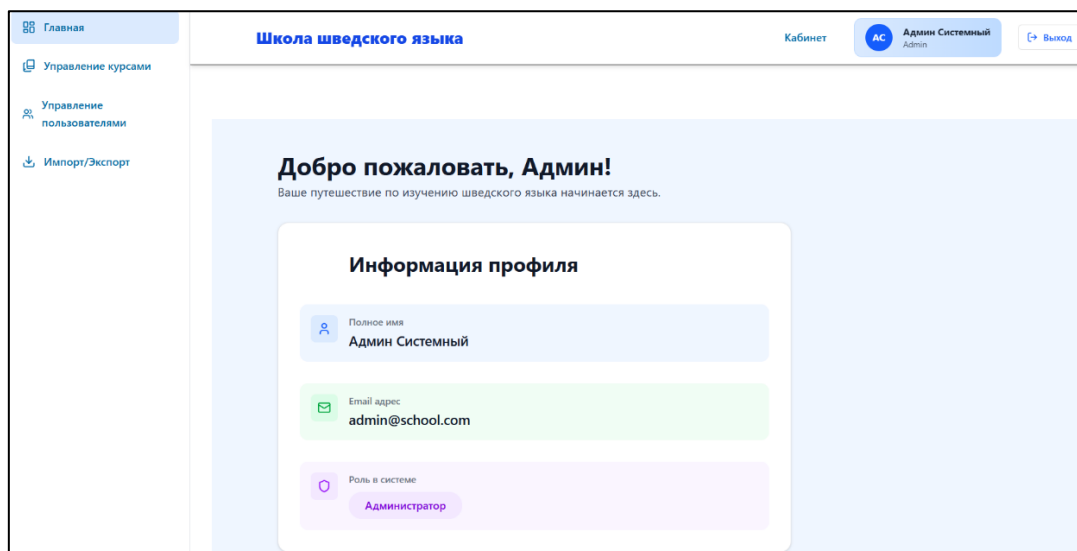


Рисунок 6 – Школа шведского языка. Вид страницы «Главная»

Для просмотра списка курсов нажать на кнопку «Управление курсами», после этого отобразится окно со списком курсов. Вид окна «Управление курсами» представлен на рисунке 7.

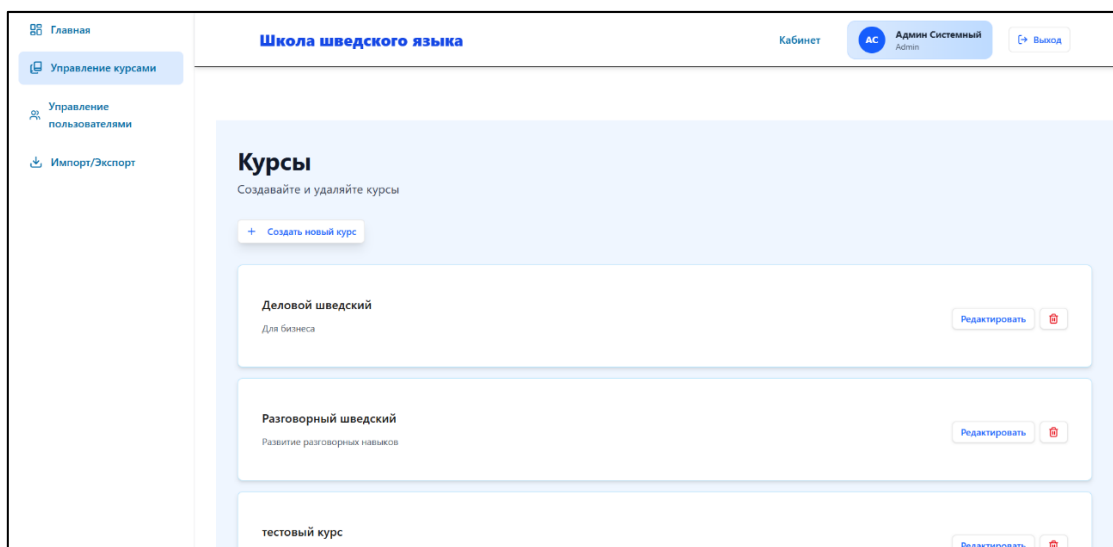


Рисунок 7 – Школа шведского языка. Вид окна «Управление курсами»

Для создания нового курса нужно нажать на кнопку «Создать новый курс», после этого отобразится форма для заполнения информации о курсе. Вид формы «Создать новый курс» представлен на рисунке 8.

The screenshot shows a web form titled "Создать новый курс" (Create new course) with a close button (X) in the top right corner. The form is set against a light blue background and contains the following fields and controls:

- Название курса** (Course name): A text input field with the placeholder text "Например: Базовый шведский" (For example: Basic Swedish).
- Описание** (Description): A larger text area with the placeholder text "Подробное описание курса..." (Detailed description of the course...).
- Уровень** (Level): A dropdown menu currently showing "Начинающий" (Beginner).
- Длительность (часы)** (Duration in hours): A text input field containing the value "20".
- Цена (₽)** (Price in rubles): A text input field containing the value "4900".
- Уроки (0)** (Lessons): A button with an upward arrow icon and the text "Уроки (0)".
- Buttons:** At the bottom right, there are two buttons: "Отмена" (Cancel) and "Создать курс" (Create course).

Рисунок 8 – Школа шведского языка. Вид формы «Создать новый курс»

Для того, чтобы экспортировать и импортировать список курсов, нужно перейти на страницу «Импорт/Экспорт» и нажать на кнопку «скачать PDF», «Выбрать файл». Пример отображения страницы «Импорт/Экспорт» показан на рисунке 9.

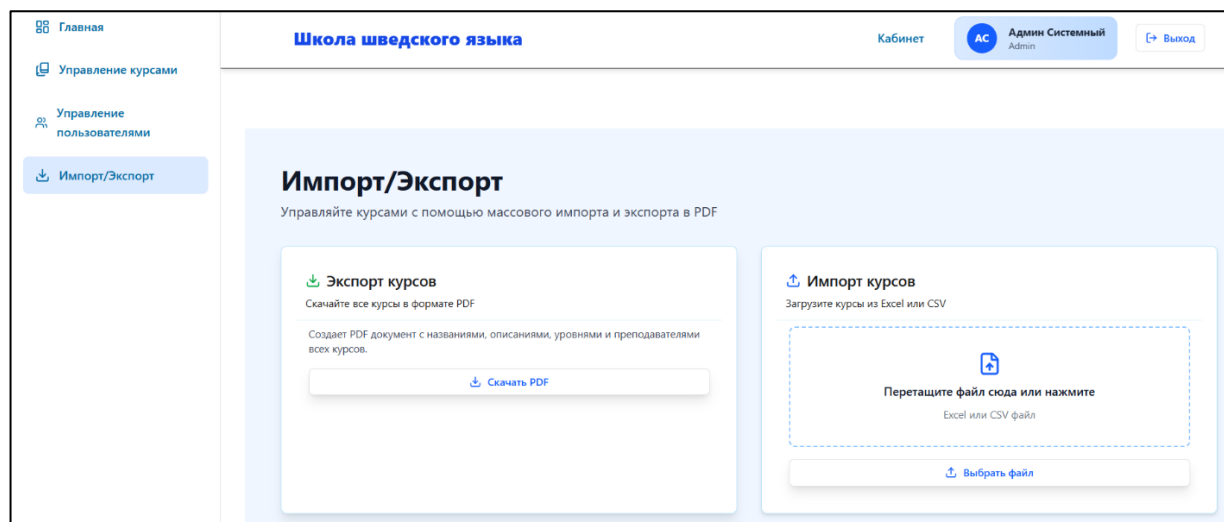


Рисунок 10 – Школа шведского языка. Вид страницы «Импорт/Экспорт»

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта достигнута поставленная цель: разработано веб-приложение для онлайн-обучения шведскому языку, обеспечивающее удобный доступ к учебным материалам, управление курсами, разграничение прав пользователей, а также запись на курсы в режиме онлайн.

В ходе курсового проектирования решены все поставленные задачи:

- проведен сбор требований целевой аудитории;
- проанализированы информационные источники по предметной области;
- спроектирована архитектура приложения;
- спроектирована диаграмма вариантов использования приложения;
- выбран состав программных и технических средств для реализации приложения;
- спроектирована БД;
- спроектирован интерфейс приложения;
- создана БД в выбранной СУБД;
- разработан API для требуемых функций приложения;
- реализовано разграничение прав доступа пользователей;
- разработан интерфейс пользователя;
- разработаны модули приложения;
- реализована работа приложения с сервером БД при помощи REST API;
- выполнено структурное и функциональное тестирование ПО;
- разработана программная и эксплуатационная документация.

В результате выполнения поставленных задач разработано приложение, отвечающее современным тенденциям разработки и требованиям заказчика.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Бек, К. Экстремальное программирование: разработка через тестирование. – Санкт-Петербург : Питер, 2021. – 224 с. – URL: <https://ibooks.ru/bookshelf/376974/reading> (дата обращения: 05.11.2024). – Режим доступа: для зарегистрир. пользователей. – Текст: электронный.

2. Гагарина, Л. Г. Технология разработки программного обеспечения : учебное пособие / Л. Г. Гагарина, Е. В. Кокорева, Б. Д. Сидорова-Виснадул ; под ред. Л. Г. Гагариной. – Москва : ФОРУМ : ИНФРА-М, 2023. – 400 с. – URL: <https://znanium.com/catalog/product/1895679> (дата обращения: 06.11.2024). – Режим доступа: по подписке. – Текст : электронный.

3. Мартишин, С. А. Базы данных. Практическое применение СУБД SQL-и NoSQL-типа для проектирования информационных систем : учебное пособие / С. А. Мартишин, В. Л. Симонов, М. В. Храпченко. — Москва : ФОРУМ : ИНФРА-М, 2023. — 368 с. - URL: <https://znanium.com/catalog/product/1912454> (дата обращения: 13.11.2024). – Режим доступа: по подписке. — Текст : электронный.

4. Тидвелл, Д. Разработка интерфейсов. Паттерны проектирования. 3-е изд. – Санкт-Петербург : Питер, 2022. – 560 с. – URL: <https://ibooks.ru/bookshelf/386796/reading> (дата обращения: 15.11.2025). – Режим доступа: для зарегистрир. пользователей – Текст : электронный.

5. Федорова, Г. Н. Разработка, внедрение и адаптация программного обеспечения отраслевой направленности : учебное пособие. — Москва : КУРС : ИНФРА-М, 2024. — 336 с. – URL: <https://znanium.ru/catalog/product/2083407> (дата обращения: 15.11.2025). – Режим доступа: по подписке. – Текст : электронный.